

SmartLead AI

Öğrenci Proje Yönergesi

Projeyi Kendiniz İnşa Edin — Adım Adım Yönerge

10 Günlük Bitirme Projesi (MVP)

Python · Flask · SQLite · Yapay Zekâ API · Wix Velo

Separation of Concerns ilkesine bağlı kurumsal mimari

Bu yönerge size kodun TAMAMINI vermez; her modülün ne yapması gerektiğini söyler. Mimari sabittir, konu sizindir: projeyi KENDİ işinize/fikrinize uyarlarsınız.

İçindekiler

İçindekiler	2
1. Projenin Amacı ve Felsefesi	3
1.1 En Önemli İlke: Sorumlulukların Ayrılığı	3
1.2 Bu Bir Şablon: Kendi İşinize Uyarlayın	3
1.3 Bu Projeyle Kanıtlayacağınız Beceriler.....	4
2. Genel Kurallar	4
3. Hedef Mimari (Ulaşmanız Gereken Yapı).....	4
4. Oturum Öncesi Hazırlık (Evde Yapılacak).....	5
5. İnşa Modülleri (Oturumda Yazacaklarınız)	6
Modül A — Yapılandırma (config.py)	6
Modül B — Veritabanı (database.py).....	6
Modül C — Yapay Zekâ Servisi (ai_service.py)	7
Modül D — Rotalar (routes.py)	7
Modül E — Fabrika ve Giriş (__init__.py + run.py).....	8
Modül F — Çalıştırma ve Test	8
Modül G — Frontend ve Wix Bağlantısı.....	9
Modül H — Yayınlama (GitHub + Render)	9
6. Önerilen 10 Günlük Zaman Çizelgesi.....	10
7. Teslim Edilecekler	11
7.1 Değerlendirme Kriterleri	11

1. Projenin Amacı ve Felsefesi

Bu projede, ziyaretçilerle yapay zekâ üzerinden sohbet eden ve onların iletişim bilgilerini (lead) toplayan **SmartLead AI** adında yeniden kullanılabilir bir sistem geliştireceksiniz. Sistem iki arayüzden oluşur: ziyaretçilerin yapay zekâ ile sohbet edip iletişim bilgisi bıraktığı bir karşılama sayfası ve işletme sahibinin bu kayıtları gördüğü bir yönetim paneli.

Bu Bir Şablondur, Konu Sizindir

SmartLead AI yeniden kullanılabilir bir İSKELETTİR: yapay zekâ ile konuşan ve lead toplayan herhangi bir işe uyarlanabilir.

Bu iskeleti kendi işinize, fikrinize veya hayalinizdeki girişime göre kişiselleştireceksiniz.

Mimari (dosya yapısı, katmanlar, kurallar) herkes için AYNIDIR. Değişen tek şey konudur: işletmenizin ne olduğu, yapay zekânın nasıl konuştuğu ve arayüzdeki metinler/marka.

Bu Yönergenin Altın Kuralı

Size her satır kod VERİLMEYECEK. Bunun yerine her modülün NE yapması gerektiğini, hangi fonksiyonları içermesi gerektiğini ve nasıl test edeceğinizi öğreneceksiniz. Kodu kendiniz yazacaksınız. Amaç sadece çalışan bir proje değil; çözümü siz üreterek öğrenmenizdir.

1.1 En Önemli İlke: Sorumlulukların Ayrılığı

Her şeyi tek bir `app.py` dosyasına yığmak (spagetti kod) **kesinlikle yasaktır**. Profesyonel yazılımda her dosyanın tek bir görevi olur:

- Ayarlar** bir dosyada, **veritabanı** başka bir dosyada.
- Yapay zekâ** çağrıları yalnızca kendi servisinde.
- Rotalar** sadece isteği karşılar ve doğru katmana yönlendirir; SQL veya AI kodu içermez.

Bu disiplin sayesinde kodunuz okunabilir, test edilebilir ve büyütülebilir olur. Değerlendirmenin en yüksek puanlı kısmı budur.

1.2 Bu Bir Şablon: Kendi İşinize Uyarlayın

İlk işiniz, projenizi hangi iş/konu üzerine kuracağınıza karar vermek. Gerçek bir işletme, bir girişim fikri ya da kişisel bir proje olabilir. Mimari aynı kalır; siz yalnızca "içeriği" değiştirirsiniz. Örnek konu fikirleri:

Örnek İş / Konu	AI Asistanı Ne Yapar?	Toplanan Lead
Restoran / kafe	Menü, rezervasyon, çalışma saatleri	Ad, telefon, kişi sayısı
Hukuk / danışmanlık	Hizmet alanları, ön görüşme yönlendirme	Ad, telefon, konu
Spor salonu	Üyelik paketleri, deneme dersi	Ad, telefon, hedef
E-ticaret / butik	Ürün önerisi, kargo, iade	Ad, telefon, ilgi alanı
Emlak ofisi	Portföy, randevu	Ad, telefon, bütçe
Eğitim / kurs	Program, kayıt yönlendirme	Ad, telefon, ilgi

Kişiselleştireceğiniz üç şey: (1) işletmenizin adı ve markası, (2) yapay zekânın kişiliğini tanımlayan `BUSINESS_CONTEXT` metni, (3) arayüzdeki başlıklar/metinler. Kod yapısı ve katmanlar aynı kalır.

BUSINESS_CONTEXT Örneği — Konuya Göre Nasıl Değişir?

Yapay zekânın "kim olduğunu" siz yazarsınız. Aynı kod, farklı bir `BUSINESS_CONTEXT` ile bambaşka bir asistana dönüşür:

```
python
```

```
# Bir kafe için:
```

```
BUSINESS_CONTEXT = """Sen Cafe Mola'nin asistanisin. Menu, fiyat ve
```

```
rezervasyon sorularini yanitla. Kibar ve samimi ol. Turkce konus.
Musteriyi rezervasyon icin iletisim birakmaya yonlendir."""
```

```
# Bir spor salonu için:
BUSINESS_CONTEXT = """Sen FitZone spor salonunun asistanisin. Uyelik
paketleri ve ders programi hakkında bilgi ver. Motive edici bir dille
uccretsiz deneme dersi icin iletisim bilgisi iste."""
```

Hatırlatma

BUSINESS_CONTEXT dışında hiçbir kodu konuya özel yazmayın. database.py, routes.py, ai_service.py gibi katmanlar her konu için aynıdır — sadece bu metin ve arayüz yazıları değişir. Mimariyi konuya gömerseniz tekrar kullanılabilirliği kaybedersiniz.

1.3 Bu Projeyle Kanıtlayacağınız Beceriler

Öğrendiğiniz Konu	Projede Nerede Kullanacaksınız
Veri tipleri, sözlükler	Lead verisini taşıma, JSON üretme
Döngüler	Veritabanı kayıtlarını işleme
Fonksiyonlar	Her katmanı fonksiyonlara bölme
Flask ile mikro sunucu	Tüm backend'in temeli
HTTP metotları (GET/POST)	Rotalarda istekleri karşılama
try-except	Hataları güvenli JSON yanıtı çevirme
Wix Velo + RESTful API	Arayüzü backend'e bağlama
Sanal ortam (venv)	Projeyi izole çalıştırma

2. Genel Kurallar

Bu kurallar projenin tamamı için geçerlidir ve değerlendirmeyi doğrudan etkiler:

Yapılması Zorunlu

- Modüler yapı:** Klasör/dosya düzeni aşağıdaki mimariye birebir uymalı.
- Sanal ortam:** Proje venv içinde çalışmalı; bağımlılıklar requirements.txt'te listelenmeli.
- Hata yönetimi:** Dış servis çağrıları (veritabanı, AI) try-except ile sarılmalı; hata kullanıcıya güvenli bir JSON olarak dönmeli.
- Güvenlik:** SQL sorgularında ? yer tutucusu kullanılmalı; API anahtarları .env içinde tutulmalı ve .gitignore'a eklenmeli.
- Yorum satırları:** Kodun kritik kısımları yorumlarla açıklanmalı.

Yapılması Yasak

- Spagetti kod:** Her şeyi tek dosyaya yığmak.
- Anahtar sızdırma:** .env dosyasını GitHub'a yüklemek.
- SQL birleştirme:** Kullanıcı verisini doğrudan SQL metnine eklemek.

Akademik Dürüstlük

Hazır şablon koddan ilham alabilirsiniz, ama her satırın ne yaptığını anlamalı ve kendi kodunuzu yazmalısınız. Sözlü değerlendirmede kendi kodunuzu açıklayamamak, projenin geçersiz sayılmasına yol açabilir. Birbirinizle fikir alışverişi serbesttir; kopya yasaktır.

3. Hedef Mimari (Ulaşmanız Gereken Yapı)

Proje tamamlandığında dosya düzeniniz aşağıdaki gibi olmalıdır. Türkçe veya İngilizce isimlendirme seçebilirsiniz; tutarlı olun.

hedef klasör yapısı

```
smartlead_ai/
├── run.py                ← Sunucuyu başlatan giriş noktası
├── config.py             ← Tüm ayarlar ve anahtarlar (.env okur)
├── requirements.txt       ← Bağımlılık listesi
├── .env                  ← Gizli anahtarlar (Git'e EKLENMEZ)
├── .gitignore
└── app/
    ├── __init__.py       ← Uygulama fabrikası (create_app)
    ├── database.py        ← Veritabanı işlemleri (SADECE burada)
    ├── routes.py          ← HTTP rotaları (sadece yönlendirme)
    ├── templates/
    │   ├── index.html     ← Karşılama sayfası
    │   └── dashboard.html ← Yönetim paneli
    └── services/
        ├── __init__.py
        └── ai_service.py  ← Yapay zekâ çağrıları (SADECE burada)
```

Mimari Sözleşme

database.py dışında HİÇBİR yerde SQL olmayacak. ai_service.py dışında HİÇBİR yerde yapay zekâ API çağırısı olmayacak. routes.py yalnızca bu iki katmanın fonksiyonlarını çağırarak. Bu kurala uyup uymadığınız, mimari puanınızın temelidir.

4. Oturum Öncesi Hazırlık (Evde Yapılacak)

Aşağıdaki **kurulum ve hesap işlerini oturuma GELMEDEN önce** tamamlayın. Bunlar mekanik işlerdir; oturumda hep birlikte yapmak değerli zamanı boşa harcar. Oturuma kod yazmaya hazır gelmelisiniz.

Yapılacaklar

1. **Araçları kurun.** Python 3.9+ (kurulumda "Add Python to PATH" işaretli), VS Code ve Git.
2. **Hesapları açın.** GitHub, Render, Wix ve yapay zekâ için **Groq** (console.groq.com) hesabı.
3. **API anahtarını alın.** Groq'tan `gsk_...` ile başlayan anahtar oluşturup güvenli bir yere kaydedin.
4. **Klasör iskeletini kurun.** Bölüm 3'teki yapıyı boş dosyalarla oluşturun (henüz kod yazmayın).
5. **Sanal ortamı kurun.** `python -m venv venv` → aktive edin → `requirements.txt`'i hazırlayıp `pip install -r requirements.txt` çalıştırın.
6. **Duman testi yapın.** Aşağıdaki 5 satırlık testi çalıştırıp tarayıcıda sonucu görün.

python

```
# hello.py – sadece ortam testi (proje dosyası DEĞİL)
from flask import Flask
app = Flask(__name__)

@app.route('/')
def merhaba():
    return 'Ortam calisiyor!'

if __name__ == '__main__':
    app.run(port=5000)
```

Terminalde `python hello.py` yazıp tarayıcıda `http://localhost:5000` açın. "Ortam calisiyor!" görürseniz hazırsınız.

Hazırım Kontrol Listesi (oturuma gelmeden işaretleyin)

#	Madde	Tamam mı?
1	python --version → 3.9+ gösteriyor	☐
2	git --version çalışıyor	☐
3	GitHub + Render + Wix + Groq hesapları açık	☐
4	Groq API anahtarı alındı ve kaydedildi	☐
5	Klasör iskeleti hazır	☐
6	venv aktif, pip install hatasız tamamlandı	☐
7	Duman testi tarayıcıda çalıştı	☐

Kanıt İsteyebiliriz

Eğitmen, duman testinin tarayıcıdaki ekran görüntüsünü oturum öncesi sizden isteyebilir. "Yaptım" demek yetmez; ortamın gerçekten çalıştığını göstermelisiniz.

5. İnşa Modülleri (Oturumda Yazacaklarınız)

Bundan sonraki modüller projenin öğretici çekirdeğidir ve oturumda eğitmen rehberliğinde yazılacaktır. Her modül size hedefi, içermesi gereken parçaları ve kontrol noktasını söyler — kodu siz yazacaksınız.

Modül A — Yapılandırma (config.py)

MODÜL A

Ayarları tek merkezde toplayın

Amaç: Tüm ayarları ve gizli anahtarları `.env` dosyasından okuyan bir yapılandırma katmanı kurmak.

İçermesi Gerekenler

- `Config` adında bir sınıf; içinde `SECRET_KEY`, `DATABASE_URL`, `GROQ_API_KEY`, `AI_PROVIDER`, `BUSINESS_CONTEXT`, `CORS_ORIGINS` gibi ayarlar.
- Her ayar `os.environ.get('ANAHTAR', 'varsayılan')` deseniyle okunmalı (anahtar yoksa güvenli varsayılan).
- Geliştirme ve üretim için iki ayrı sınıf (`DEBUG=True/False`) ve bunları seçen bir sözlük.
- BUSINESS_CONTEXT:** Yapay zekânın kişiliğini tanımlayan metin. **Burayı kendi işinize göre yazın** (örn. "Sen [işletmenizin adı]'nın asistanısın...").

İpucu

python-dotenv kütüphanesinin `load_dotenv()` fonksiyonunu dosyanın başında çağırmayı unutmayın; aksi halde `.env` okunmaz.

✓ Kontrol Noktası

`config.py`'yi import edip bir ayarı yazdırabiliyorsanız ve `.env`'deki değer geliyorsa modül tamamdır.

Modül B — Veritabanı (database.py)

MODÜL B

Tüm veri işlemlerini izole edin

Amaç: SQLite ile çalışan, müşteri adaylarını kaydeden ve listeleyen veri katmanı. SQL yalnızca burada olacak.

İçermesi Gereken Fonksiyonlar

Fonksiyon	Görevi
get_db()	Veritabanına bağlanır; satırlara sütun adıyla erişim sağlar
init_db(app)	'leads' tablosunu oluşturur (yoksa)
lead_ekle(isim, telefon, mesaj)	Yeni kayıt ekler
tum_leadler()	Tüm kayıtları en yeniden eskiye getirir

Tablo Şeması (leads)

- id (otomatik artan), isim (zorunlu), telefon (zorunlu), mesaj (opsiyonel), tarih (otomatik zaman).
- İsteğe bağlı kişiselleştirme:** İşinize uygun ek bir sütun ekleyebilirsiniz (örn. restoran için `kisi_sayisi`, emlak için `butce`). Eklerseniz fonksiyonları ve formu da buna göre güncelleyin.

Güvenlik Zorunluluğu

lead_ekle içinde değerleri SQL metnine doğrudan eklemeyin. ? yer tutucusu ve ayrı parametre kullanın. Bu, SQL Injection'a karşı zorunlu korumadır.

✓ Kontrol Noktası

Modül F'de test edeceksiniz: bir kayıt ekleyip listeleyebiliyorsanız tamamdır. Şimdilik fonksiyonların tam ve SQL'in sadece burada olduğundan emin olun.

Modül C — Yapay Zekâ Servisi (ai_service.py)

MODÜL C

Yapay zekâ çağrılarını izole edin

Amaç: Kullanıcı mesajını alıp seçilen yapay zekâ servisine (Groq) gönderen, yanıtı döndüren servis katmanı. AI çağrıları yalnızca burada olacak.

İçermesi Gerekenler

- Bir `AIService` sınıfı ve içinde bir `yanit_uret(mesaj, gecmis)` metodu.
- Sistem talimatını (`BUSINESS_CONTEXT`) config'den okuyan bir yardımcı metod.
- Groq API'sine `requests.post` ile istek atan bir metod (model: `llama-3.1-8b-instant`).
- Anahtar yoksa çökmek yerine bir "demo modu" mesajı döndürme.
- Servise özel bir hata sınıfı (`AIServiceError`) ve dosya sonunda tek bir örnek (`ai_service = AIService()`).

İpucu

Groq'a gönderilen 'messages' dizisi şu sırayla olmalı: önce sistem talimatı (role: system), sonra geçmiş mesajlar, en sonda yeni kullanıcı mesajı. İstek try-except ile sarılmalı; hata olursa `AIServiceError` fırlatın.

Mimari Hatırlatma

Bu dosya Flask, HTTP veya veritabanı kavramlarını BİLMEZ. Sadece yapay zekâ ile konuşur. Bu izolasyon, ileride sağlayıcı değiştirmeyi (Groq → Gemini) tek satıra indirir.

✓ Kontrol Noktası

Modül F'de test edilecek: gerçek bir soruya yapay zekâdan Türkçe yanıt dönüyorsa tamamdır.

Modül D — Rotalar (routes.py)

MODÜL D

HTTP isteklerini karşılayıp yönlendirin

Amaç: Gelen HTTP isteklerini karşılayan, doğrulayan ve doğru katmana yönlendiren kontrolcü. Burada SQL veya AI kodu OLMAYACAK.

İçermesi Gereken Uç Noktalar

Metod + Yol	Görevi	Hangi katmanı çağırır?
GET /	Karşılama sayfasını gösterir	render_template
GET /dashboard	Yönetim panelini gösterir	render_template
POST /api/sohbet	AI'a mesaj iletir	ai_service.yanıt_uret()
POST /api/leads	Yeni lead kaydeder	database.lead_ekle()
GET /api/leads	Tüm lead'leri getirir	database.tüm_leadler()

Kurallar

- İki **Blueprint** kullanın: biri API (/api önekiyle), biri sayfalar için.
- Her API uç noktası `basari` alanı içeren bir JSON döndürmeli.
- Eksik veri gelirse 400, AI hatası olursa 503, yeni kayıtta 201 durum kodu kullanın.
- AI çağırısını `try-except` ile sarın; `AIServiceError` yakalayıp kibar JSON hata dönün.

✓ Kontrol Noktası

routes.py içinde hiç SQL/AI kodu olmamalı; sadece import edilen fonksiyonlar çağrılmalı. Beş uç nokta da tanımlı olmalı.

Modül E — Fabrika ve Giriş (__init__.py + run.py)

MODÜL E

Tüm parçaları birleştirin

Amaç: Ayarları, CORS'u, veritabanını ve rotaları bir araya getiren bir fabrika fonksiyonu ve onu çalıştıran giriş dosyası.

__init__.py İçermesi Gerekenler

- Bir `create_app()` fonksiyonu.
- Sırasıyla: ayarları yükle → CORS aç → `init_db()` çağır → blueprint'leri kaydet → uygulamayı döndür.
- Bir `/health` uç noktası (sunucu canlılık kontrolü).

run.py İçermesi Gerekenler

- Fabrikadan uygulamayı al (`app = create_app()`) ve `if __name__ == '__main__':` bloğunda çalıştır.

İpucu

`init_db()` bir uygulama bağlamı (app context) içinde çağrılmalı: `with app.app_context(): ...` Blueprint'leri kaydederken API'ye `url_prefix='/api'` verin.

✓ Kontrol Noktası

python run.py çalışıyor ve hata vermiyorsa modül tamamdır. Bir sonraki modülde test edeceğiz.

Modül F — Çalıştırma ve Test

MODÜL F

Backend'in çalıştığını kanıtlayın

Amaç: Tüm backend'i çalıştırıp uç noktaları test etmek. Bu, projenin en kritik dönüm noktasıdır.

Test Adımları

7. Sunucuyu başlatın: `python run.py`.
8. Tarayıcıda `/health` açın; canlılık JSON'u dönmeli.
9. Sohbeti test edin: `/api/sohbet`'e POST atın (curl veya bir araçla); yapay zekâdan yanıt gelmeli.
10. Lead kaydını test edin: `/api/leads`'e POST atın.
11. Listeyi kontrol edin: tarayıcıda `/api/leads` açın; eklediğiniz kayıt görünmeli.

✓ Kontrol Noktası

`/health`, `/api/sohbet` ve `/api/leads` uçtan uca çalışıyorsa BACKEND TAMAMEN HAZIR demektir. Projenin en zor kısmını geçtiniz.

Modül G — Frontend ve Wix Bağlantısı

MODÜL G

Arayüzü backend'e bağlayın

Amaç: İki arayüz oluşturmak ve sizin API'nize bağlamak. Proje gereği **Wix Velo** hedeflenir. Arayüzdeki tüm metinler, renkler ve marka **kendi işinizi** yansıtmalı (logo, başlık, slogan, sektör kartları).

Arayüz 1 — Karşılama Sayfası (B2C)

- Bileşenler: mesaj girişi, "Sor" butonu, yanıt alanı, isim/telefon girişleri, "Kaydet" butonu — hepsine ID verin.
- UX deseni: **Z-Pattern**. Logo sol üst, sohbet sağ, form gözün indiği alt bölgede. Sohbet kartına **Glassmorphism** (buzlu cam) efekti.
- Velo kodu `wix-fetch` ile `/api/sohbet` ve `/api/leads`'e istek atmalı.

Arayüz 2 — Yönetim Paneli (B2B)

- Bir **Repeater** bileşeni ile lead'leri tablo gibi gösterin.
- UX deseni: **F-Pattern**. En önemli kolon (isim) solda.
- GET `/api/leads`'ten gelen veriyi Repeater'a bağlayın; her objede `_id` zorunlu, satır içinde `$item` kullanın.

KRİTİK — Aynı Kelimeler

Frontend'in gönderdiği alan adları backend ile birebir aynı olmalı. Sohbet için mesaj/cevap, kayıt için isim/telefon, liste için backend'in döndürdüğü anahtar kullanın. Bir harf farkı bağlantıyı koparır.

✓ Kontrol Noktası

Karşılama sayfasında sohbet ve form çalışıyor, yönetim panelinde kayıtlar görünüyorsa modül tamamdır.

Modül H — Yayınlama (GitHub + Render)

MODÜL H

Projeyi canlıya alın

Amaç: Kodu GitHub'a yüklemek ve Render'da herkesin erişebileceği bir adreste yayınlamak.

Adımlar

12. Git deposu başlatın, commit alın (`.env` yüklenmemeli!).

13. GitHub'da public bir depo açıp kodu push edin.
14. Render'da "Web Service" oluşturup deponuzu bağlayın.
15. Build: `pip install -r requirements.txt` · Start: `gunicorn run:app`.
16. Ortam değişkenlerini (GROQ_API_KEY, SECRET_KEY vb.) Render'a ekleyin.
17. Wix kodundaki API adresini Render adresinizle güncelleyin.

KRİTİK Kontrol

GitHub deponuzda .env GÖRÜNMEMELİ. Görünüyorsa anahtarınızı hemen yenileyin ve .gitignore'u düzeltin.

✓ Kontrol Noktası

Render adresinizde /health "aktif" dönüyor ve Wix sayfanız bu adrese bağlanıp çalışıyorsa PROJE TAMAMLANDI.

6. Önerilen 10 Günlük Zaman Çizelgesi

Gün	Hedef	Modül
Öncesi	Ortam + hesaplar + iskelet (evde)	Bölüm 4
1	Yapılandırma	Modül A
2	Veritabanı	Modül B
3-4	Yapay zekâ servisi	Modül C
5	Rotalar	Modül D
6	Fabrika + giriş	Modül E
7	Çalıştırma + test	Modül F
8	Frontend / Wix	Modül G
9	GitHub + Render	Modül H
10	Düzeltilme, README, sunum hazırlığı	Teslim

7. Teslim Edilecekler

Proje sonunda aşağıdakileri teslim etmelisiniz:

- **GitHub deposu bağlantısı** — tüm kod, `.env` hariç.
- **Canlı Render adresi** — çalışan backend.
- **Wix site bağlantısı** — yayınlanmış iki arayüz.
- **Kısa README** — projenin ne yaptığı ve nasıl çalıştırılacağı.
- **Kısa sunum/demo** — sistemi tanıtan birkaç dakikalık anlatım.

7.1 Değerlendirme Kriterleri

Kriter	Ağırlık	Beklenti
Mimari (SoC)	%30	Her dosya tek sorumluluk; katmanlar temiz ayrılmış
Çalışırılık	%25	Backend + frontend uçtan uca sorunsuz
Kod Kalitesi	%15	Okunaklı, yorumlu, anlamlı isimlendirme
Güvenlik	%10	? ile SQL koruması, <code>.env</code> gizli, CORS ayarlı
Hata Yönetimi	%10	try-except ile kibar hata yanıtları
Yayın + Sunum	%10	Canlıda çalışıyor; öğrenci kodunu açıklayabiliyor

Başarı İçin Son Tavsiyeler

Her modülün kontrol noktasını atlamadan ilerleyin; bir modül çalışmadan diğerine geçmeyin.

Kodunuzu yorumlayın — hem değerlendirmede hem de sözlü savunmada işinize yarar.

Hata mesajını dikkatle okuyun; çözümün yarısı oradadır.

Bu mimariyi içselleştirseniz, bundan sonra her web projesini aynı disiplinle kurabilirsiniz. Hedefiniz tek bir projeden çok daha büyük olsun.